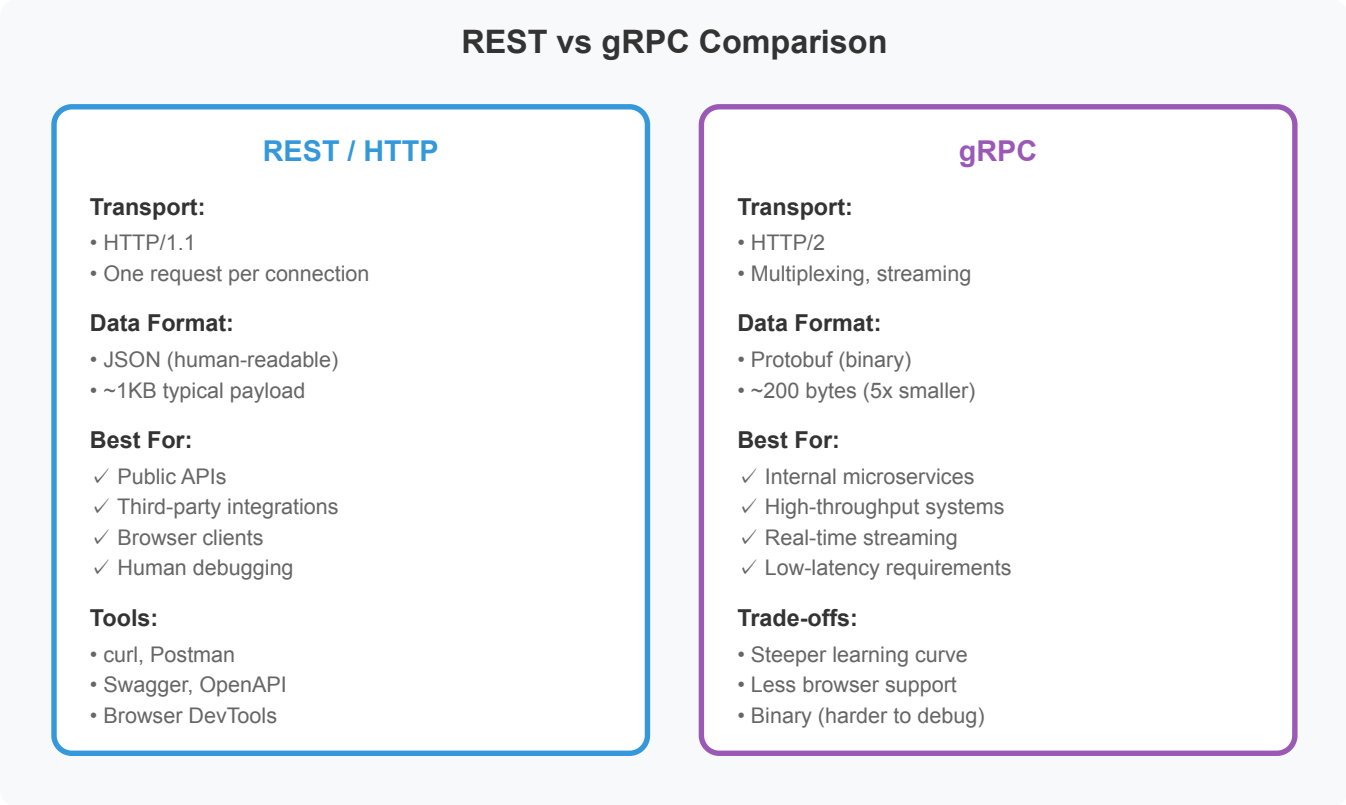
Implementing Synchronous Communication

systemdesignschool.io/fundamentals/synchronous-communication-implementation



We explored synchronous communication and its trade-offs. When you decide synchronous calls fit your use case, two protocols dominate: REST and gRPC. Each has different strengths and ideal scenarios.

REST

REST (Representational State Transfer) is the default choice for most web APIs. It uses HTTP methods like GET, POST, PUT, and DELETE to perform operations on resources. Data typically travels as JSON, though XML and other formats work too.

A client requests user data with a GET request:

```
GET /users/123 HTTP/1.1
Host: example.com
```

The server processes the request and returns a response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 123,
  "name": "John Doe"
}
```

REST is widely adopted because it builds on HTTP, which every platform supports. Browsers, mobile apps, and IoT devices all speak HTTP. This ubiquity means libraries and tools exist for every programming language. Developers can test REST APIs with curl or Postman without special setup.

The human-readable format helps during development and debugging. You can read a JSON response and understand what it contains. Browser developer tools display REST calls clearly. Documentation tools like Swagger generate interactive API explorers automatically.

REST works best for public-facing APIs and external integrations. Third-party developers can understand and consume your API without learning new protocols. The stateless nature of REST means each request contains all necessary information. The server does not need to remember previous requests.

gRPC

gRPC (gRPC Remote Procedure Call) prioritizes performance over simplicity. It uses HTTP/2 for transport and Protocol Buffers for serialization. The binary format reduces payload size compared to JSON. HTTP/2 enables features like multiplexing and bi-directional streaming.

In gRPC, you define services in a `.proto` file:

```
service UserService {
  rpc GetUser (UserRequest) returns (UserResponse);
}

message UserRequest {
  int32 id = 1;
}

message UserResponse {
  int32 id = 1;
  string name = 2;
}
```

A client makes a call that looks like a local function:

```
response = stub.GetUser(UserRequest(id=123))
print(response.name)  # Outputs "John Doe"
```

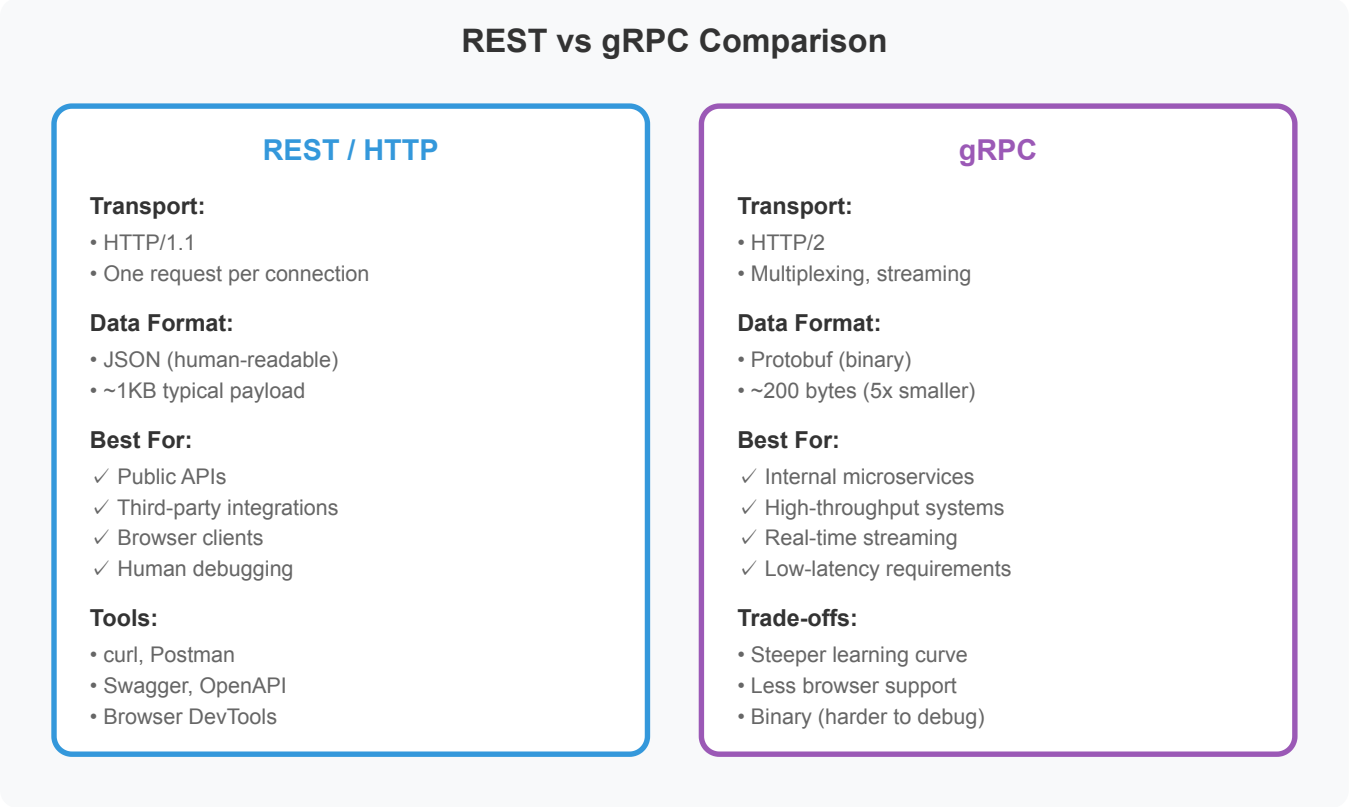
Protocol Buffers serialize data more efficiently than JSON. A JSON object with nested arrays might be 1KB. The equivalent Protobuf message could be 200 bytes. This reduction matters for high-throughput systems transferring large amounts of data.

gRPC supports streaming in both directions. A server can push updates to a client without the client polling. A client can stream data to a server without waiting for each chunk to be acknowledged. This bidirectional streaming works well for real-time features.

The strong typing catches errors at compile time. If you change a field type in the proto file, code that uses the old type fails to compile. JSON APIs only catch type errors at runtime when the actual request fails.

gRPC excels in internal service-to-service communication where performance matters and both ends are controlled by the same team. Microservices within a company can benefit from reduced latency and smaller payloads. The learning curve and tooling requirements make it less suitable for public APIs.

Choosing Between REST and gRPC



Use REST for external APIs and public-facing services. Third-party developers expect REST. They have tools for it. JSON is easy to understand. Documentation is straightforward.

Use gRPC for internal microservice communication where performance is critical. If services transfer large amounts of data or make thousands of calls per second, the efficiency gains matter. If you need streaming, gRPC provides it natively.

Many systems use both. REST APIs face external clients. gRPC handles internal service mesh communication. The API gateway translates between them, exposing REST externally while using gRPC internally.